

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序序设计

批改人：

实验	Pthreads 并行构造	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	

1. 实验目的

使用此前构造的 `parallel_for` 并行结构，将 `heated_plate_openmp` 改造为基于 Pthreads 的并行应用。

2. 实验过程和核心代码

在之前实验的 `parallel_for` 实现代码的基础上进行补充，添加动态调度和引导调度。动态调度使得线程不断竞争获取以 `chunk_size` 为单位的迭代量的工作进行执行，直到执行完所有的迭代任务。引导调度会依据当前的剩余工作量不断地动态减小 `chunk_size` 的大小，以保证不会出现线程因为尾部剩余任务过少导致分配不均匀。目标是使得的所有的线程负载均衡。核心代码如下：

```
// 动态调度 - 线程动态获取下一个可用的块
else if (data->schedule_type == SCHEDULE_DYNAMIC) {
    int chunk_size = data->chunk_size > 0 ? data->chunk_size : 1;
    int next_i;

    while (1) {
        pthread_mutex_lock(data->mutex);
        next_i = *data->next_index;
        *data->next_index = next_i + chunk_size;
        pthread_mutex_unlock(data->mutex);

        if (next_i >= data->end) break;

        int end_i = next_i + chunk_size;
        if (end_i > data->end) end_i = data->end;

        for (int i = next_i; i < end_i; i += data->inc) {
            data->func(i, data->args);
        }
    }
}
```

```

// 引导调度 - 块大小随着执行逐渐减小
else if (data->schedule_type == SCHEDULE_GUIDED) {
    int min_chunk = data->chunk_size > 0 ? data->chunk_size : 1;
    int next_i;

    while (1) {
        pthread_mutex_lock(data->mutex);
        next_i = *data->next_index;

        // 计算引导块大小 - 剩余工作量除以线程数
        int remaining = data->end - next_i;
        int chunk = (remaining + data->num_threads - 1) / data->num_threads; // 向上取整

        if (chunk < min_chunk) chunk = min_chunk;
        if (chunk > remaining) chunk = remaining;

        *data->next_index = next_i + chunk;
        pthread_mutex_unlock(data->mutex);

        if (next_i >= data->end) break;
        | Ctrl+L to chat, Ctrl+K to generate
        int end_i = next_i + chunk;
        if (end_i > data->end) end_i = data->end;

        for (int i = next_i; i < end_i; i += data->inc) {
            data->func(i, data->args);
        }
    }
}

```

然后实现对主程序的改写，改写遵循之前的流程，将源程序中的代码进行改写，对于其中使用 **critical** 确保同时只有一个线程执行的部分我采用线程锁来实现互斥。改写后如下：

```

#include "parallel_for.h"
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <pthread.h>
#include <sys/time.h>

#define M 500
#define N 500

// 获取当前时间 (秒)
double get_wtime() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1e6;
}

typedef struct {
    double (*w)[N];      // 当前温度数组
    double (*u)[N];      // 上一次迭代的温度数组
    double mean;          // 平均值
    double diff;          // 最大差异
    pthread_mutex_t mutex; // 互斥锁
} HeatedPlateData;

```

```

// 边界初始化函数
void init_left(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    data->w[i][0] = 100.0;
}

void init_right(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    data->w[i][N-1] = 100.0;
}

void init_bottom(int j, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    data->w[M-1][j] = 100.0;
}

void init_top(int j, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    data->w[0][j] = 0.0;
}

// 计算左右边界温度和
void sum_left_right(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    // pthread_mutex_lock(&data->mutex);
    data->mean += data->w[i][0] + data->w[i][N-1];
    // pthread_mutex_unlock(&data->mutex);
}

// 计算上下边界温度和
void sum_top_bottom(int j, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    // pthread_mutex_lock(&data->mutex);
    data->mean += data->w[M-1][j] + data->w[0][j];
    // pthread_mutex_unlock(&data->mutex);
}

```



```

// 初始化内部节点函数
void init_internal(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    double mean = data->mean;
    for (int j = 1; j < N-1; j++) {
        data->w[i][j] = mean;
    }
}

// 复制数组函数
void copy_u(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    for (int j = 0; j < N; j++) {
        data->u[i][j] = data->w[i][j];
    }
}

// 计算新温度函数 - 使用正确的热传导公式
void compute_w(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    for (int j = 1; j < N-1; j++) {
        data->w[i][j] = (data->u[i-1][j] + data->u[i+1][j] +
                        data->u[i][j-1] + data->u[i][j+1]) / 4.0;
    }
}

```

```

// 计算差异函数
void compute_diff(int i, void *args) {
    HeatedPlateData *data = (HeatedPlateData *)args;
    double my_diff = 0.0;

    for (int j = 1; j < N-1; j++) {
        double current_diff = fabs(data->w[i][j] - data->u[i][j]);
        if (my_diff < current_diff) {
            my_diff = current_diff;
        }
    }

    pthread_mutex_lock(&data->mutex);
    if (data->diff < my_diff) {
        data->diff = my_diff;
    }
    pthread_mutex_unlock(&data->mutex);
}

```

```

// 初始化数据结构
HeatedPlateData data;
data.w = w;
data.u = u;
data.mean = 0.0;
data.diff = 0.0;
pthread_mutex_init(&data.mutex, NULL);

// 设置边界条件
parallel_for_schedule(1, M-1, 1, init_left, &data, num_threads, schedule_type, chunk_size);
parallel_for_schedule(1, M-1, 1, init_right, &data, num_threads, schedule_type, chunk_size);
parallel_for_schedule(0, N, 1, init_bottom, &data, num_threads, schedule_type, chunk_size);
parallel_for_schedule(0, N, 1, init_top, &data, num_threads, schedule_type, chunk_size);

// 计算初始平均值
parallel_for_schedule(1, M-1, 1, sum_left_init, &data, num_threads, schedule_type, chunk_size);
parallel_for_schedule(0, N, 1, sum_right_init, &data, num_threads, schedule_type, chunk_size);
data.mean = data.mean / (double)(2 * M + 2 * N - 4);

printf("\n");
printf(" 边界平均温度 MEAN = %f\n", data.mean);

// 初始化内部节点
parallel_for_schedule(1, M-1, 1, init_internal, &data, num_threads, schedule_type, chunk_size);

// 迭代计算
printf("\n");
printf(" 迭代次数 变化量\n");
printf("\n");

double wtime = get_wtime();
data.diff = epsilon;

```

```

while (epsilon <= data.diff) {
    // 重置差异值
    data.diff = 0.0;

    // 保存旧值
    parallel_for_schedule(0, M, 1, copy_u, &data, num_threads, schedule_type, chunk_size);

    // 计算新值
    parallel_for_schedule(1, M-1, 1, compute_w, &data, num_threads, schedule_type, chunk_size);

    // 计算差异
    parallel_for_schedule(1, M-1, 1, compute_diff, &data, num_threads, schedule_type, chunk_size);

    iterations++;
    if (iterations == iterations_print) {
        printf(" %8d %f\n", iterations, data.diff);
        iterations_print = 2 * iterations_print;
    }
}

double elapsed = get_wtime() - wtime;

printf("\n");
printf(" %8d %f\n", iterations, data.diff);
printf("\n");
printf(" 误差容限已达到\n");
printf(" 耗时 = %f 秒\n", elapsed);

// 运行OpenMP版本进行对比 (如果可用)
printf("\n");
printf("性能对比:\n");
printf("  Pthreads版本 (线程数=%d, 调度类型=%d, 块大小=%d): %f 秒, 迭代次数: %d\n",
    num_threads, schedule_type, chunk_size, elapsed, iterations);

```

编译运行结果如下：

```

● cake@CAK:~/并行程序设计/hw7/Heated-plate-参考资料$ gcc -fPIC -shared -o libparallel.so parallel_for.c
● cake@CAK:~/并行程序设计/hw7/Heated-plate-参考资料$ gcc -o diy diy.c -L. -lparallel -lpthread -Wl,-rpath=.
● cake@CAK:~/并行程序设计/hw7/Heated-plate-参考资料$ ./diy 20 0

```

HEATED_PLATE_PTHREADS

使用parallel_for实现的Pthreads版本
求解平板稳态温度分布问题

空间网格: 500 x 500 点
迭代将重复直到变化 $\leq 1.000000e-03$
线程数: 20
调度类型: 0 (静态)
块大小: 0

边界平均温度 MEAN = 74.949900

迭代次数 变化量

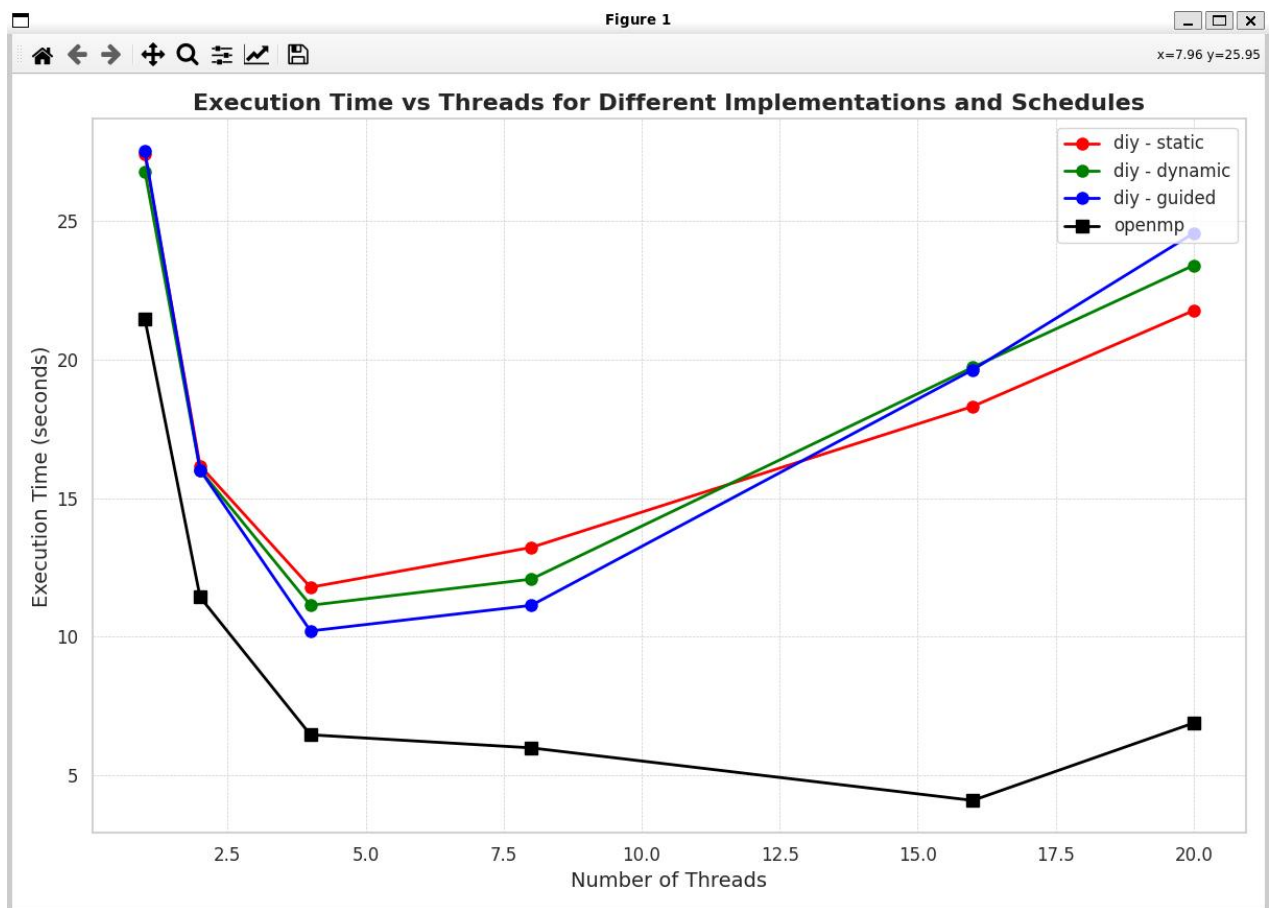
1	18.737475
2	9.368737
4	4.098823
8	2.289577
16	1.136604
32	0.568201
64	0.282805
128	0.141777
256	0.070808
512	0.035427
1024	0.017707
2048	0.008856
4096	0.004428
8192	0.002210
16384	0.001043

16955	0.001000
-------	----------

误差容限已达到
耗时 = 24.133761 秒

3. 实验结果

编写脚本来实现数据的采集，稍微修改实验给定代码，使其可以接受外部传入的线程数目参数。分别以不同的线程数，调度方式来运行，收集数据到 csv 文件，并通过 python 脚本绘图展示，结果如下：



具体数据如下：

A	B	C	D
threads	schedule	implementation	time
1		0 diy	27.407718
1		1 diy	26.786147
1		2 diy	27.513377
2		0 diy	16.165355
2		1 diy	16.011877
2		2 diy	16.010605
4		0 diy	11.778991
4		1 diy	11.129321
4		2 diy	10.202811
8		0 diy	13.216896
8		1 diy	12.072769
8		2 diy	11.125585
16		0 diy	18.307939
16		1 diy	19.718955
16		2 diy	19.631208
20		0 diy	21.766275
20		1 diy	23.397227
20		2 diy	24.557297
1	NA	openmp	21.459309
2	NA	openmp	11.424616
4	NA	openmp	6.454987
8	NA	openmp	5.982933
16	NA	openmp	4.086048
20	NA	openmp	6.879734

对比数据可以发现，自定义实现的程序之间相差并不大，变化趋势都是一样的，推测是因为任务本身负载均衡的原因，导致三种调度方式没有明显的区别。在4-8个线程时运行效率最高，随着线程数上升，性能不升反降。和原程序 OpenMP 版本实现进行对比，可以发现使用 Pthread 实现的性能是不如 OpenMP 实现的，并且 OpenMP 随着线程数增加性能下降现象出现的也比手动实现版本出现的晚。分析其原因，对比线程数为1时，会发现两者本身就有性能差距，我手动实现了一个串行部分作为对照，其运行结果如下：

```

● cake@CAK:~/并行程序设计/hw7/Heated-plate-参考资料$ ./a

HEATED_PLATE_SERIAL
Solving the steady state heat equation on a 2D plate.
Grid size: 500 x 500
Iteration continues until the max change <= 1.000000e-03
Initial mean value = 74.949900

Iteration  Change
      1  18.737475
      2   9.368737
      4   4.098823
      8   2.289577
     16   1.136604
     32   0.568201
     64   0.282805
    128   0.141777
    256   0.070808
    512   0.035427
   1024   0.017707
   2048   0.008856
   4096   0.004428
   8192   0.002210
  16384   0.001043

  16955   0.001000

Tolerance achieved.
Elapsed time = 20.314948 seconds

HEATED_PLATE_SERIAL:
Normal end of execution.

```

可以发现其和 OpenMP 一个线程时是一致的，由此推测这多余的开销是因为 Pthread 自身带来的额外开销导致的。如线程的创建与销毁等。

除此之外，我在网上查找资料，发现 OpenMP 采用了线程池的实现机制，有效减少了线程创建与销毁的开销，而我目前的实现是在每一次任务开始时都会新创建线程，并在每一次任务完成后销毁。由此推测，Pthread 和 OpenMP 的主要性能差异就是线程的创建与销毁开销方面，这也解释了为什么 OpenMP 出现线程数增加，性能下降的现象要晚于 Pthread。

4. 实验感想

通过这次实验，我将前一次实验的模型进行补充，并且应用于任务要求，收集对比了不同调度方式和线程数的性能差异，并分析了与 OpenMP 的性能对比，分析两者之间性能差异主要是线程创建和销毁开销导致。通过这次实验，我受益匪浅。